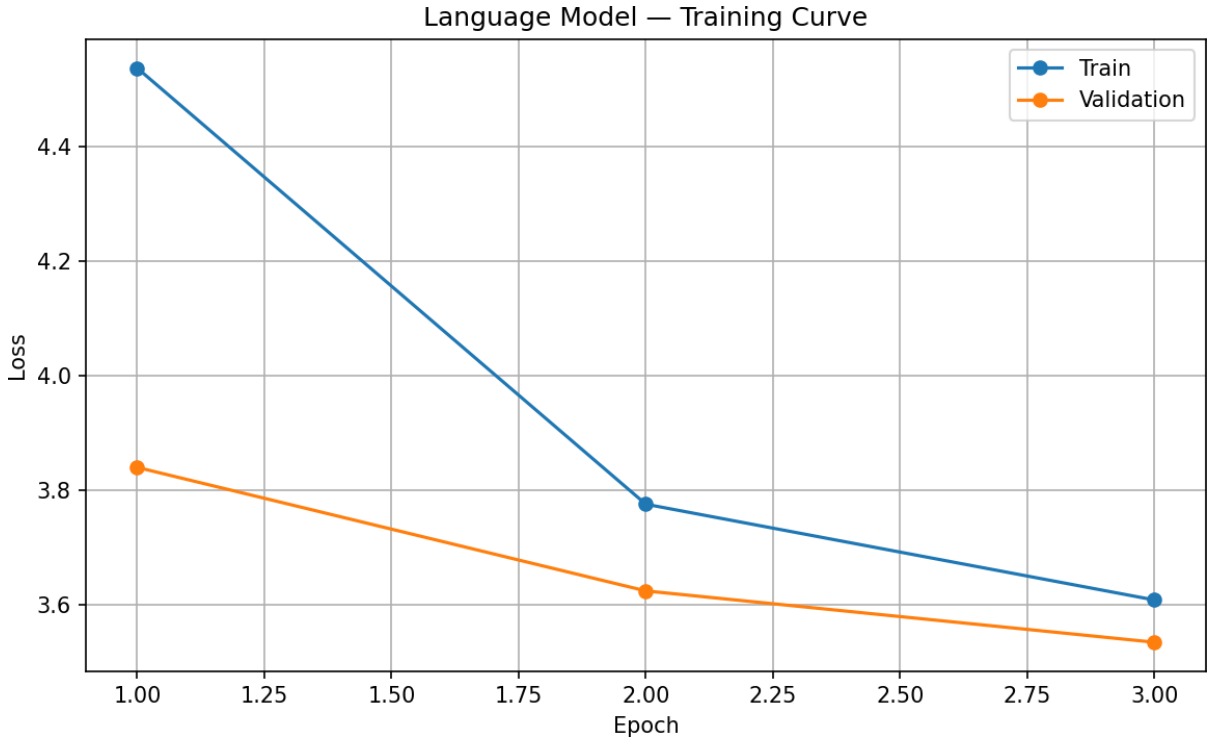
**Decoder Only GPT 2 Architecture (Hindi Version)**

**Plots and Metrics** -

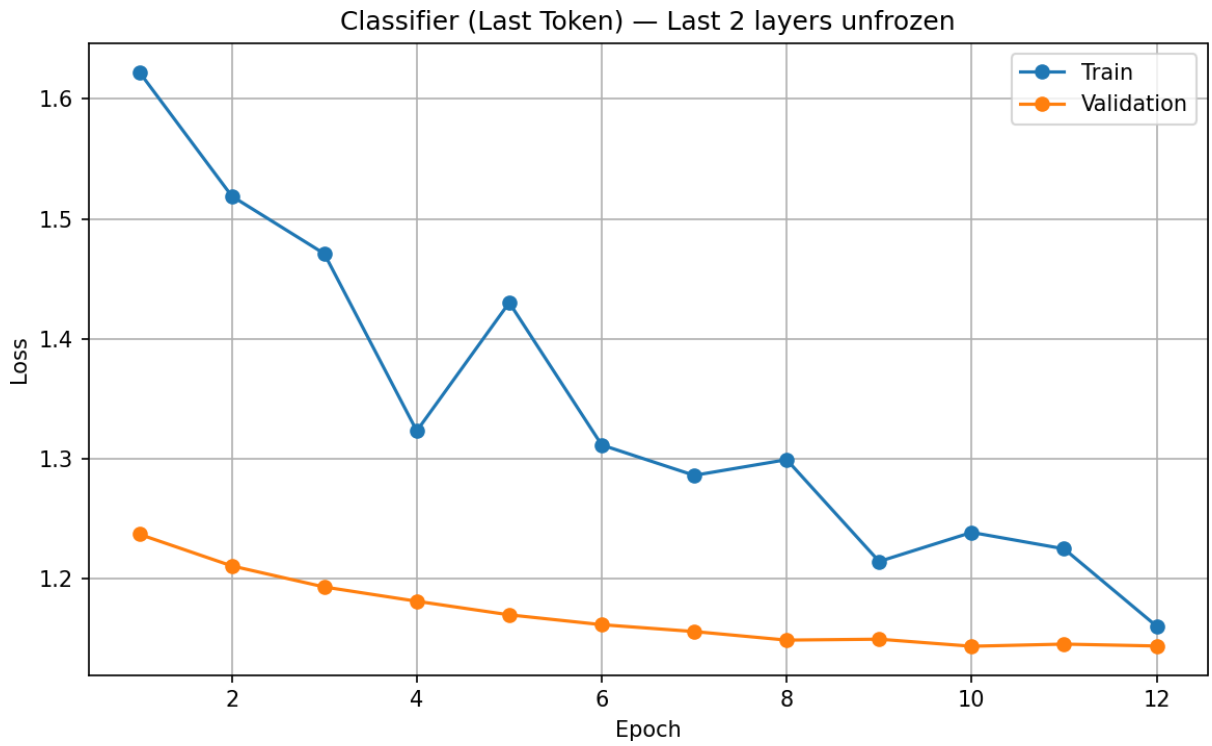
Task 2 - Training GPT-2 Model : We ran the model for 3 Epochs. We had 4 Transformer Layers, In each Layer we did attention 10 times. Time to run the code was approx 38 mins. **The Final Perplexity we ended up with was => 33.36 (Lower the perplexity better the model).** The backbone is a decoder-only Transformer with learned token and positional embeddings, 10 Transformer blocks, residual connections, layer normalization, and a feed-forward network with ReLU activation. The hidden size is 256 with dropout 0.1, and causal masking is enforced through scaled dot-product attention. The language-model head is a linear projection to vocabulary size with tied token-embedding/output weights. Training used AdamW,

batch size 64, learning rate 3e-4, and stride 64 for overlapping windows.




Task 3 - Training the Classifier : **The final validation accuracy we ended up with was 45.83**. For Task 3, a classification head was added on top of the pretrained GPT backbone. The classifier uses the hidden state of the last token, which is appropriate for decoder-only sequence classification. The model was trained with dropout 0.5, label smoothing 0.1, weight decay 0.05, cosine annealing scheduling, and early stopping with patience 7 => In actuality the model was supposed to run for 30 epochs. The last two Transformer blocks were unfrozen for adaptation to the sentiment task. We didn't train the entire model as if we kept all the weights unfreezed off the backbone and the linear classifier head, there would be too many parameters and we could risk over-fitting as the number of data points was also small. Also, the backbone was already trained so we didn’t think there was any point in updating all the weights. Hence, we decided to go for a technique like PEFT (Parameter Efficient Fine Tuning). We unfreezed the first two layers of transformer weights and the last linear layer and freezed rest of the backbone - Logic behind this was this way we could also modify

the existing GPT-2 to also incorporate some of the classification behaviour in its thinking. A suitable in between to save time and prevent over-fitting.




**Some Design Choices** -

a) For preprocessing, we first created explicit train/validation splits for both the Hindi corpus and the classification dataset, since the handout requires separate splits before training. We trained a SentencePiece BPE tokenizer with a vocabulary size of 5,000 only on the training portion of the corpus to avoid data leakage from the validation set. The same tokenizer was then reused across both the language modeling and classification tasks so that the input representation remained consistent throughout the pipeline.

b) In Task 2 during the construction of training samples for next-word prediction. Each example was formed as an input sequence x and a shifted target sequence y, exactly as required in the handout. We also used overlapping windows from the token stream, but with a stride greater than 1 during training. This was done to reduce the total number of training examples and speed up training, while still

preserving the language modeling objective. For optimization, we used AdamW and gradient clipping to improve stability, especially because Transformer training can otherwise become noisy or unstable.

c) Several regularization choices in Task 3 were implemented to improve generalization on the smaller classification dataset. Rather than unfreezing the full backbone immediately, we fine-tuned only the final part of the pretrained model so that useful language features were retained while still allowing adaptation to sentiment classification. We added dropout in the classifier head, used weight decay, and applied label smoothing to reduce overconfidence. Early stopping and a learning-rate scheduler were also used so that training could stop once validation performance stopped improving, which helps avoid overfitting.